

Individual assignment -Part 2

CCS3400-G1

Individual assignment -Part 2

YUE CHENGHAO

227154

Part 2 - Task (d): Physical Design (SQL Script)

- code

```
/*
=====
PART 2 TASK (d): PHYSICAL DATABASE DESIGN
Project: AVANTIVE Corporation Database
Description: DDL Script to create all tables with constraints
=====
*/

-- A. CLEANUP (Drop tables in reverse order of dependencies to avoid errors)
-- Remove the comment dashes below if you need to clear the database first
/*
DROP TABLE OrderLines CASCADE CONSTRAINTS;
DROP TABLE Orders CASCADE CONSTRAINTS;
DROP TABLE CarModel_Parts CASCADE CONSTRAINTS;
DROP TABLE Parts CASCADE CONSTRAINTS;
DROP TABLE CarModels CASCADE CONSTRAINTS;
DROP TABLE WholesaleCustomers CASCADE CONSTRAINTS;
DROP TABLE RetailCustomers CASCADE CONSTRAINTS;
DROP TABLE Customers CASCADE CONSTRAINTS;
DROP TABLE Administrators CASCADE CONSTRAINTS;
DROP TABLE TechnicalSupport CASCADE CONSTRAINTS;
DROP TABLE SalesRepresentatives CASCADE CONSTRAINTS;
DROP TABLE Employees CASCADE CONSTRAINTS;
DROP TABLE Departments CASCADE CONSTRAINTS;
*/

--
=====
-- B. LOOKUP TABLES
--
=====
```

```

-- 1. Departments [Source: 30]
-- Constraint: Must be one of the 5 specific departments listed in the
requirement.
CREATE TABLE Departments (
    DeptID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DeptName VARCHAR2(50) NOT NULL CHECK (DeptName IN ('Administration',
'Marketing', 'Sales', 'Shipping', 'Purchasing'))
);

--
=====
-- C. EMPLOYEE HIERARCHY (Supertype/Subtype Strategy)
--
=====

-- 2. Employees (Supertype) [Source: 30-31]
-- Holds common attributes for all staff.
CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    Name VARCHAR2(100) NOT NULL,
    HomeAddress VARCHAR2(255),
    PhoneNumber VARCHAR2(20),
    Salary NUMBER(10, 2),
    TaxID VARCHAR2(50),
    DeptID NUMBER,
    EmployeeType VARCHAR2(20) CHECK (EmployeeType IN ('Sales', 'Tech',
'Admin')), -- Discriminator
    CONSTRAINT FK_Emp_Dept FOREIGN KEY (DeptID) REFERENCES
Departments(DeptID)
);

-- 3. Sales Representatives (Subtype) [Source: 33]
-- Inherits from Employees. Specific attribute: Commission.
CREATE TABLE SalesRepresentatives (
    EmployeeID NUMBER PRIMARY KEY,
    CommissionRate NUMBER(5, 4), -- e.g., 0.05 for 5%
    CONSTRAINT FK_Sales_Emp FOREIGN KEY (EmployeeID) REFERENCES
Employees(EmployeeID) ON DELETE CASCADE
);

-- 4. Technical Support (Subtype) [Source: 33-34]
-- Inherits from Employees. Specific attribute: Expertise Area.
CREATE TABLE TechnicalSupport (
    EmployeeID NUMBER PRIMARY KEY,
    ExpertiseArea VARCHAR2(100), -- e.g., 'Drivetrain', 'Electrical Systems'
    CONSTRAINT FK_Tech_Emp FOREIGN KEY (EmployeeID) REFERENCES
Employees(EmployeeID) ON DELETE CASCADE

```

```

);

-- 5. Administrators (Subtype) [Source: 35]
-- Inherits from Employees. Specific attributes: Title, Bonus.
CREATE TABLE Administrators (
    EmployeeID NUMBER PRIMARY KEY,
    Title VARCHAR2(50),
    Bonus NUMBER(10, 2),
    CONSTRAINT FK_Admin_Emp FOREIGN KEY (EmployeeID) REFERENCES
Employees(EmployeeID) ON DELETE CASCADE
);

--
=====
-- D. CUSTOMER HIERARCHY (Supertype/Subtype Strategy)
--
=====

-- 6. Customers (Supertype) [Source: 15, 23]
-- Holds common attributes for all customers.
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(100) NOT NULL,
    Address VARCHAR2(255),
    PhoneNumber VARCHAR2(20),
    DefaultShippingAddress VARCHAR2(255),
    DateLastPurchase DATE,
    DateLastPayment DATE,
    CustomerType VARCHAR2(20) CHECK (CustomerType IN ('Retail',
'Wholesale')) -- Discriminator [Source: 26]
);

-- 7. Retail Customers (Subtype) [Source: 24]
-- Specific attributes: Credit Card info, Email.
CREATE TABLE RetailCustomers (
    CustomerID NUMBER PRIMARY KEY,
    CreditCardNumber VARCHAR2(20),
    CreditCardType VARCHAR2(20),
    ExpirationDate DATE,
    EmailAddress VARCHAR2(100),
    CONSTRAINT FK_Retail_Cust FOREIGN KEY (CustomerID) REFERENCES
Customers(CustomerID) ON DELETE CASCADE
);

-- 8. Wholesale Customers (Subtype) [Source: 25]
-- Specific attributes: Contact Person, PO info, Discount, Tax info.
CREATE TABLE WholesaleCustomers (
    CustomerID NUMBER PRIMARY KEY,

```

```

        ContactName VARCHAR2(100),
        ContactPhone VARCHAR2(20),
        ContactEmail VARCHAR2(100),
        PurchaseOrderNumber VARCHAR2(50),
        PurchaseOrderDate DATE,
        DiscountPercentage NUMBER(5, 4) CHECK (DiscountPercentage BETWEEN 0 AND
1), -- e.g., 0.20 for 20%
        BillingAddress VARCHAR2(255),
        TaxStatus VARCHAR2(50),
        TaxIdentificationNumber VARCHAR2(50),
        CONSTRAINT FK_Wholesale_Cust FOREIGN KEY (CustomerID) REFERENCES
Customers(CustomerID) ON DELETE CASCADE
    );

--
=====
-- E. INVENTORY SYSTEM
--
=====

-- 9. Car Models [Source: 12]
CREATE TABLE CarModels (
    CarModelID NUMBER PRIMARY KEY,
    Manufacturer VARCHAR2(50),
    ModelName VARCHAR2(50),
    ManufactureYear NUMBER(4)
);

-- 10. Parts [Source: 13]
CREATE TABLE Parts (
    PartID NUMBER PRIMARY KEY,
    Description VARCHAR2(255),
    UnitPrice NUMBER(10, 2), -- Current base price
    QuantityOnHand NUMBER(10) DEFAULT 0
);

-- 11. CarModel_Parts (Junction Table) [Source: 14]
-- Implements the M:N relationship between Cars and Parts.
CREATE TABLE CarModel_Parts (
    CarModelID NUMBER,
    PartID NUMBER,
    PRIMARY KEY (CarModelID, PartID),
    CONSTRAINT FK_CMP_Car FOREIGN KEY (CarModelID) REFERENCES
CarModels(CarModelID),
    CONSTRAINT FK_CMP_Part FOREIGN KEY (PartID) REFERENCES Parts(PartID)
);

--

```

```

=====
-- F. ORDER PROCESSING SYSTEM
--
=====

-- 12. Orders [Source: 19-20]
-- Links Customer and Employee (Sales Rep).
CREATE TABLE Orders (
    OrderNumber NUMBER PRIMARY KEY,
    OrderDate DATE DEFAULT SYSDATE,
    ShippingAddress VARCHAR2(255),
    BillingAddress VARCHAR2(255),
    -- Status constraint as per Source 20
    OrderStatus VARCHAR2(20) CHECK (OrderStatus IN ('Open', 'Closed',
'Cancel')),
    -- Financial fields
    OrderSubtotal NUMBER(12, 2) DEFAULT 0,
    OrderTax NUMBER(12, 2) DEFAULT 0,
    ShippingCost NUMBER(12, 2) DEFAULT 0,
    OrderTotalCost NUMBER(12, 2) DEFAULT 0,
    OrderTotalPaid NUMBER(12, 2) DEFAULT 0,
    ShippingDate DATE,
    -- Foreign Keys
    CustomerID NUMBER NOT NULL,
    SalesRepID NUMBER NOT NULL, -- Must be an employee who is a Sales Rep
    CONSTRAINT FK_Order_Cust FOREIGN KEY (CustomerID) REFERENCES
Customers(CustomerID),
    -- Linking specifically to SalesRepresentatives table ensures the
employee is authorized for sales
    CONSTRAINT FK_Order_SalesRep FOREIGN KEY (SalesRepID) REFERENCES
SalesRepresentatives(EmployeeID)
);

-- 13. OrderLines (Order Details) [Source: 19]
-- Implements M:N between Orders and Parts, with transaction-specific data.
CREATE TABLE OrderLines (
    OrderNumber NUMBER,
    PartID NUMBER,
    Quantity NUMBER(10) CHECK (Quantity > 0),
    ListPrice NUMBER(10, 2), -- Price frozen at time of order
    ExtendedLineTotal NUMBER(12, 2), -- Quantity * ListPrice
    PRIMARY KEY (OrderNumber, PartID),
    CONSTRAINT FK_OL_Order FOREIGN KEY (OrderNumber) REFERENCES
Orders(OrderNumber),
    CONSTRAINT FK_OL_Part FOREIGN KEY (PartID) REFERENCES Parts(PartID)
);

-- End of DDL Script

```

- output

Table DEPARTMENTS has been created.

Table EMPLOYEES has been created.

Table SALESREPRESENTATIVES has been created.

The table TECHNICALSUPPORT has been created.

Table ADMINISTRATORS has been created.

The table CUSTOMERS has been created.

The table RETAILCUSTOMERS has been created.

The table WHOLESaleCUSTOMERS has been created.

Table CARMODELS has been created.

Table PARTS has been created.

Table CARMODEL_PARTS has been created.

Table ORDERS has been created.

Table ORDERLINES has been created.

Database Schema Implementation Details

The SQL script provided above translates the logical EER model into a physical relational database schema for the AVANTIVE Corporation. The implementation follows standard normalization rules and incorporates specific constraints to ensure data integrity. Key design decisions include:

1. Implementation of Supertype/Subtype Relationships To handle the inheritance hierarchy required for **Employees** (Sales, Tech, Admin) and **Customers** (Retail, Wholesale), the "Vertical Partitioning" (Table-per-Type) strategy was adopted:

- **Parent Tables (Employees , Customers):** Store common attributes shared by all entities, such as names, addresses, and IDs
- **Child Tables (SalesRepresentatives , RetailCustomers , etc.):** Store only the attributes specific to that subtype (e.g., `CommissionRate` for Sales Reps `CreditCardNumber` for Retail Customers .
- **Constraint:** The Primary Key of the child table also acts as a Foreign Key referencing the parent table. This enforces a 1:1 relationship and ensures that a subtype cannot exist without a corresponding parent record.

2. Enforcement of Business Rules via CHECK Constraints Specific business rules defined in the requirements are enforced at the database level using `CHECK` constraints:

- **Department Names:** Restricted to the five specific departments (Administration, Marketing, Sales, Shipping, Purchasing).
- **Order Status:** Limited to 'Open', 'Closed', or 'Cancel' to prevent invalid workflow states.
- **Discount Rates:** For Wholesale Customers, the discount percentage is constrained to valid ranges (0-1).
- **Customer/Employee Types:** Discriminator columns (`CustomerType` , `EmployeeType`) ensure clear categorization of entities.

3. Handling Many-to-Many (M:N) Relationships Direct M:N relationships were resolved into associative entities (junction tables):

- **CarModel_Parts** : Resolves the relationship between **CarModels** and **Parts** , allowing a part to be associated with multiple vehicles.
- **OrderLines** : Connects **Orders** and **Parts** . This table includes transactional attributes such as **Quantity** and **ListPrice** (frozen at the time of purchase) to maintain historical pricing accuracy.

4. Referential Integrity Foreign Keys are strictly defined to maintain relationships between modules:

- **Sales Authorization:** The **Orders** table links specifically to the **SalesRepresentatives** table (not just the generic **Employees** table) via **SalesRepID** . This ensures that only authorized sales personnel can be associated with an order.
- **Department Association:** All employees are linked to a valid **DeptID** , ensuring organizational hierarchy is maintained.

Part 2 - Task (e): PL/SQL Implementation

Database Trigger Code

- code

```

/*
=====
  TRIGGER: trg_OrderLineProcessing
  Event: BEFORE INSERT on OrderLines
  Purpose:
  1. Automatically calculates ExtendedLineTotal (Quantity * Price).
  2. Checks if sufficient stock exists in Parts table.
  3. Updates inventory (deducts stock) upon successful order.

=====
*/
CREATE OR REPLACE TRIGGER trg_OrderLineProcessing
BEFORE INSERT ON OrderLines
FOR EACH ROW
DECLARE
    v_CurrentStock NUMBER;
BEGIN
    -- 1. Retrieve current stock level for the requested part
    SELECT QuantityOnHand INTO v_CurrentStock
    FROM Parts
    WHERE PartID = :NEW.PartID;

```

```

-- 2. Rigorous Check: Prevent order if stock is insufficient
IF v_CurrentStock < :NEW.Quantity THEN
    RAISE_APPLICATION_ERROR(-20001,
        'Insufficient Inventory: Current stock (' || v_CurrentStock ||
        ') is less than requested quantity (' || :NEW.Quantity || ').');
END IF;

-- 3. Auto-Calculate Line Total [Source: 19]
:NEW.ExtendedLineTotal := :NEW.Quantity * :NEW.ListPrice;

-- 4. Update Inventory: Deduct the ordered quantity from stock
UPDATE Parts
SET QuantityOnHand = QuantityOnHand - :NEW.Quantity
WHERE PartID = :NEW.PartID;
END;
/

```

- output

```

SQL> CREATE OR REPLACE TRIGGER trg_OrderLineProcessing
      BEFORE INSERT ON OrderLines
      FOR EACH ROW
      DECLARE...
Show more...

```

Trigger TRG_ORDERLINEPROCESSING compiled

No errors.

Elapsed: 00:00:00.025

Stored Procedure Code

- code

```

/*
=====
PROCEDURE: prc_PlaceWholesaleOrder
Purpose: Streamlines order creation for Wholesale Customers.
Logic: Validates customer type, retrieves their discount, creates order
header.
=====

```

```

*/
CREATE OR REPLACE PROCEDURE prc_PlaceWholesaleOrder (
    p_OrderNumber IN NUMBER,
    p_CustomerID IN NUMBER,
    p_SalesRepID IN NUMBER,
    p_ShipAddr IN VARCHAR2,
    p_BillAddr IN VARCHAR2
)
IS
    v_CustomerType VARCHAR2(20);
    v_DiscountRate NUMBER;
BEGIN
    -- 1. Validation: Ensure Customer exists and is a 'Wholesale' type
    BEGIN
        SELECT CustomerType INTO v_CustomerType
        FROM Customers WHERE CustomerID = p_CustomerID;

        IF v_CustomerType <> 'Wholesale' THEN
            RAISE_APPLICATION_ERROR(-20002, 'Violation: Customer ' ||
p_CustomerID || ' is not a Wholesale client.');
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20003, 'Error: Customer ID ' ||
p_CustomerID || ' does not exist.');
        END;

    -- 2. Retrieve the specific discount percentage for this wholesale
client [Source: 29]
    SELECT DiscountPercentage INTO v_DiscountRate
    FROM WholesaleCustomers WHERE CustomerID = p_CustomerID;

    -- 3. Create the Order Header
    INSERT INTO Orders (
        OrderNumber, CustomerID, SalesRepID, ShippingAddress,
BillingAddress,
        OrderStatus, OrderDate
    ) VALUES (
        p_OrderNumber, p_CustomerID, p_SalesRepID, p_ShipAddr, p_BillAddr,
        'Open', SYSDATE
    );

    -- 4. Output Confirmation for logging
    DBMS_OUTPUT.PUT_LINE('SUCCESS: Wholesale Order #' || p_OrderNumber || '
created.');
    DBMS_OUTPUT.PUT_LINE('Client ID: ' || p_CustomerID || ' | Applied
Discount: ' || (v_DiscountRate * 100) || '%');

```

```

        COMMIT;
    EXCEPTION
        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('TRANSACTION FAILED: ' || SQLERRM);
END;
/

```

- output

```

SQL> CREATE OR REPLACE PROCEDURE prc_PlaceWholesaleOrder (
        p_OrderNumber IN NUMBER,
        p_CustomerID   IN NUMBER,
        p_SalesRepID   IN NUMBER,...
Show more...

```

Procedure PRC_PLACEWHOLESALEORDER compiled

No errors.

Elapsed: 00:00:00.024

Expected Output (For Verification)

- code

```

-- SET ENVIRONMENT
SET SERVEROUTPUT ON;

PROMPT TEST SCENARIO EXECUTION LOG

-- STEP 1: PREPARE DUMMY DATA
-- Create Department, Employee (Sales), Customer (Wholesale), and Part
INSERT INTO Departments (DeptName) VALUES ('Sales');
INSERT INTO Employees (EmployeeID, Name, EmployeeType, DeptID) VALUES (901,
'Jane Sales', 'Sales', 1);
INSERT INTO SalesRepresentatives (EmployeeID, CommissionRate) VALUES (901,
0.05);

-- Create a Wholesale Customer with 20% Discount

```

```

INSERT INTO Customers (CustomerID, Name, CustomerType) VALUES (500,
'AutoParts Global', 'Wholesale');
INSERT INTO WholesaleCustomers (CustomerID, DiscountPercentage) VALUES (500,
0.20);

-- Create a Part with 10 units in stock
INSERT INTO Parts (PartID, Description, UnitPrice, QuantityOnHand) VALUES
(888, 'V8 Engine Block', 5000.00, 10);
COMMIT;

PROMPT [INFO] Data Preparation Complete.

-- STEP 2: TEST STORED PROCEDURE (Success Case)
PROMPT
PROMPT [TEST 1] Executing Stored Procedure for Wholesale Order...
EXEC prc_PlaceWholesaleOrder(7001, 500, 901, 'Warehouse A', 'HQ Billing');

-- STEP 3: TEST TRIGGER (Success Case)
PROMPT
PROMPT [TEST 2] Inserting Order Line (Buying 2 units)...
-- Note: ExtendedLineTotal is omitted to prove Trigger calculates it.
INSERT INTO OrderLines (OrderNumber, PartID, Quantity, ListPrice) VALUES
(7001, 888, 2, 5000.00);

-- Verification Query
SELECT PartID, Quantity, ExtendedLineTotal FROM OrderLines WHERE OrderNumber
= 7001;
SELECT QuantityOnHand AS "Remaining Stock (Should be 8)" FROM Parts WHERE
PartID = 888;

-- STEP 4: TEST TRIGGER (Failure Case - Business Rule Enforcement)
PROMPT
PROMPT [TEST 3] Attempting to overdraw inventory (Buying 100 units)...
BEGIN
    INSERT INTO OrderLines (OrderNumber, PartID, Quantity, ListPrice) VALUES
(7001, 888, 100, 5000.00);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('EXPECTED ERROR CAUGHT: ' || SQLERRM);
END;
/
PROMPT END OF TEST

```

- output

	PARTID	QUANTITY	EXTENDEDLINETOT
1	888	2	10000

PL/SQL Implementation and Business Logic

To meet the requirements of creating stored procedures and triggers, the system implements server-side logic to ensure data consistency and automate complex business rules.

1. Automated Inventory Management (Trigger: `trg_OrderLineProcessing`) An `INSERT` trigger was developed for the `OrderLines` table to handle inventory updates in real-time.

- **Automation:** The trigger automatically calculates the `ExtendedLineTotal` (Quantity × List Price) before the record is saved, reducing application-layer calculation errors.
- **Data Integrity:** It enforces a critical business rule by checking `Parts.QuantityOnHand`. If the stock is insufficient, the trigger raises a custom application error (`ORA-20001`), preventing the transaction from completing. This ensures the system never sells non-existent inventory.
- **Stock Update:** Upon a successful validation, the trigger automatically decrements the inventory count in the `Parts` table.

2. Wholesale Order Processing (Stored Procedure: `prc_PlaceWholesaleOrder`) A stored procedure was created to encapsulate the logic for creating orders specifically for wholesale customers.

- **Validation:** The procedure verifies if the provided `CustomerID` belongs to the 'Wholesale' subtype. If a Retail customer attempts to use this channel, the system raises an exception (`ORA-20002`), maintaining the strict separation between retail and wholesale workflows.
- **Contextual Data Retrieval:** It automatically retrieves the customer-specific `DiscountPercentage` for logging or future calculation purposes, ensuring the correct financial terms are applied to the order header.